의료 실험 실행 가이드 (통합본)

작성일: 2026-11-07 · 요청: Zero PII 팀 (발표 자료 결과 ③·⑥ 갱신용)

이 가이드는 이전에 보낸 "결과 ⑥ 30 명 측정 가이드"와 "공개율 실행 가이드"를 대체합니다. 두 목적의 실행을 한 번에 처리합니다.

1. 무엇을 얻어야 하는지

목적	필요한 것	실행
결과 ③ 개인정보 공개율 재계산	방어 없음(C0), 단일 응답 방어(C1S), 멀티턴 방어(C1)가 같은 질의로 돌아간 턴 기록	실행 A, B
결과 ⑥ 응답 시간 (30 명)	턴별 시간(sec_answer), 순서를 바꾼 두 블록	실행 A, B
기준 민감도 (교수님 질문)	일반화·억제 기준을 바꾼 C1 턴 기록	실행 S1, S2

2. 준비

1. 저장소를 받고 브랜치 `feat/medical-integration` 을 씁니다. 현재 최신 커밋은 `0b44873` 입니다. 이전 가이드에 적었던 `2e6cda9` 는 저장소에서 찾을 수 없었으므로 쓰지 않습니다.

   ```
   git clone -b feat/medical-integration
   https://github.com/ZEROPII/ZEROP_PII.git
   cd ZEROP_PII
   ```

2. 타이밍 패치를 먼저 적용합니다. 턴별 시간과 git 커밋 해시를 기록하도록 고치는 패치입니다 (`01_턴시간_기록_패치.patch`).

   ```
   git apply --check 01_턴시간_기록_패치.patch && git apply 01_턴시간_
   기록_패치.patch
   ```

   적용 후 `medical/session.py` 와 `medical/experiment.py` 가 바뀝니다. 실행 결과의 판정 로직은 바뀌지 않습니다.

3. 패키지를 설치합니다 (저장소 `README.md` 절차).

4. `.env` 를 만듭니다. 키는 본인 것을 직접 넣습니다. **`.env` 는 압축하거나 보내지 않습니다.**

   ```
   LLM_MODEL=claude-haiku-4-5-20251001
   ```

5. 기준 집단을 점검합니다 (LLM 호출 없음).

   ```
   python -m medical audit
   ```

3. 소규모 확인 (먼저 한 번)

비용과 시간이 크므로 본 실행 전에 확인합니다.

```
python -m medical run --arms C1,C1S,C0 --limit 2 --mode exploration --
budget 12 --out runs/medical/확인_소규모
```

확인할 것: - `runs/medical/확인_소규모/턴기록.jsonl` 에서 턴 1 이상의 줄에 `sec_answer` 가 숫자로 있는지 - `run_meta.json` 에 `git_commit`(해시)과 `machine` 이 있는지 - `trial 요약.json` 에 `LLM_ERROR` 가 없는지

문제가 있으면 에러 메시지와 함께 알려 주세요. 본 실행은 확인 후에 합니다.

4. 본 실행

실행 **A:** 순서 **C0 → C1**

```
python -m medical run --arms C0,C1S,C1 --limit 30 --mode exploration
--budget 12 --out runs/medical/A_C0_C1S_C1
```

실행 **B:** 순서 **C1 → C0** (순서 효과를 줄이기 위해 반대로)

```
python -m medical run --arms C1,C1S,C0 --limit 30 --mode exploration
--budget 12 --out runs/medical/B_C1_C1S_C0
```

실행 **S1:** 기준 민감도 (일반화 **10**, 억제 **5**)

```
python -m medical run --arms C1S,C1 --limit 30 --mode exploration --
budget 12 --generalize-threshold 10 --suppress-threshold 5 --out
runs/medical/S1_g10_s5
```

실행 **S2:** 기준 민감도 (일반화 **20**, 억제 **10**)

```
python -m medical run --arms C1S,C1 --limit 30 --mode exploration --
budget 12 --generalize-threshold 20 --suppress-threshold 10 --out
runs/medical/S2_g20_s10
```

설명: - A 와 B 는 같은 질의 환자 30 명(`--limit 30`)을 씁니다. 질의 환자 선택은 시드로 고정되어 있습니다. - 세 갈래의 의미: C0 = 방어 없음, C1S = 단일 응답 방어(답변 하나씩 검사), C1 = 멀티턴 방어(대화 전체 누적으로 검사). 슬라이드 33 쪽의 세 줄과 같습니다. - S1 과 S2 는 기준에 따라 달라지는 C1S 와 C1 만 돌립니다. C0 는 기준과 무관하므로 A 의 C0 결과를 기준선으로 씁니다. - 기준 기본값(5, 3)은 A 와 B 의 C1 결과가 그대로 기준입니다. 저장소 기본값은 바꾸지 않습니다. - 각 실행 폴더 이름은 위 명령대로 써 주세요. 집계 스크립트가 폴더 이름으로 구분합니다.

5. 보내 주실 것

각 실행 폴더를 통째로 압축해서 보내 주세요. `.env` 는 제외합니다.

```
zip -r A_C0_C1S_C1.zip runs/medical/A_C0_C1S_C1
zip -r B_C1_C1S_C0.zip runs/medical/B_C1_C1S_C0
zip -r S1_g10_s5.zip runs/medical/S1_g10_s5
zip -r S2_g20_s10.zip runs/medical/S2_g20_s10
```

필수 파일: 턴기록.jsonl, trial 요약.json, run_meta.json, report.md, llm_usage.jsonl

함께 적어 주실 것: - 각 실행의 시작·종료 시각 (대략이라도) - 중간에 멈추거나 에러가 난 경우 메시지 - 실행 중 다른 무거운 작업이 있었는지 (시간 측정에 영향)

6. 주의

- API 비용과 시간이 큽니다. 갈래가 세 개로 늘어서 A 와 B 는 이전 계획보다 약 1.5 배 듭니다. 소규모 확인 후 A, B, S1, S2 를 순서대로 돌립니다.
- 실행 A 와 B 는 같은 질의로 두 번 도는 것이므로 시간대를 다르게 해 주시면 좋습니다 (예: 오전 A, 오후 B).
- 타이밍 패치는 저장소에 push 하지 않습니다. 로컬에서만 적용해 주세요.

7. 개발자가 할 분석과 산출물

실험을 돌린 뒤 분석까지 개발자가 맡아 주세요. 같이 넣은 tools/ 폴더의 두 스크립트는 참고 구현입니다. 그대로 써도 되고, 저장소 구조에 맞게 옮겨도 됩니다. 어떤 방식이든 계산 정의는 아래 명세를 따릅니다.

7-1. 공개율 (결과 ③)

- 항목 단위: 답변 안의 개인 단서 하나 = release_plans 의 decisions 한 줄 (C1, C1S). C0 는 raw_observations 의 관찰 한 개 = 그대로 공개 한 개.
- 그대로 공개 = action 이 ALLOW, 일반화 공개 = GENERALIZE, 비공개 = SUPPRESS.
- 비율 = 각 행동 수 ÷ 전체 항목 수. 일반화는 그대로율에 넣지 않습니다.
- 속성 묶음: 개인정보 = AGE, SEX, REGION, PERIOD / 의료정보 = DX, PROC, MED, PLAN.
- 누적 그대로율: trial 안에서 턴 t 까지 한 번이라도 그대로 나간 항목 ÷ 턴 t 까지 나온 전체 항목 (항목 식별자 = 사례, 속성, 노드).

7-2. 속성별 k 감소량 (결과 ③ 보강)

- 항목 하나의 감소량 = $\log_2$(k_before ÷ k_raw). k_before 는 그 사례의 공개 전 k(release_plans 의 plan), k_raw 는 그 항목을 원래 값으로 넣었을 때의 k(decision).
- k_raw 가 0 이면 따로 세고, 평균에는 넣지 않습니다.
- 속성별 평균 감소량, 감소량 중앙값, 그리고 k_raw ≤ 5 인 항목의 비율을 냅니다.
- C0 는 release_plans 가 없으므로, 턴마다 cases 의 k 가 바뀐 턴에서 새로 추가된 속성이 하나뿐일 때만 $\log_2$(이전 k ÷ 이번 k)를 그 속성에 붙입니다. 여러 속성이 동시에 추가되면 세지 않고 개수만 보고합니다.

7-3. 응답 시간 (결과 ⑥)

- 턴 1 이상의 sec_answer 만 씁니다. 턴 0 은 갈래 간 공유라서 뺍니다.
- 갈래별 중앙값, IQR, p95, p95 의 95% 신뢰구간 (질의 환자 단위 부트스트랩, 1000 회, 시드 7).
- C1 − C0 차이는 두 갈래 모두 있는 환자만 짝지어 계산합니다.
- 실행 A 와 B 를 함께 넣습니다.

7-4. 기준 민감도

- S1, S2 의 C1S 와 C1 결과를 A 의 기본 기준(5, 3)과 나란히 놓습니다.
- 기준별 그대로율, 일반화율, 비공개율, 평균 유용성을 한 표로 만듭니다.

산출물 (개발자가 보내 주실 것)

1. 분석 코드: 저장소 밖 별도 폴더나 파일 (저장소에는 push 하지 않습니다)
2. 결과 파일: `disclosure_summary.md`(속성별·턴별 공개율), `kdrop_summary.md`(속성별 k 감소량, 새로 만든 파일), `latency_summary.md`(응답 시간), `sweep_summary.md`(기준별 비교)
3. 각 결과를 만든 명령과 커밋 해시 (`run_meta.json` 의 `git_commit`)
4. 검증 메모: 7-2 의 감소량을 한두 턴에서 손으로 계산한 값과 맞춘 기록, 그리고 `report.md` 의 위반율과 공개 방식 집계가 턴 기록에서 직접 센 값과 같은지 확인한 기록

8. 나와야 하는 결과 체크리스트

아래 항목이 모두 채워져야 이번 작업이 끝난 것입니다. 빠진 항목이 있으면 이유를 함께 적어 주세요.

파일	반드시 들어가야 할 내용	확인 기준
`disclosure_summary.md`	갈래(C0, C1S, C1)별 속성별 그대로·일반화·비공개 수와 비율, 개인정보·의료정보 묶음 합계, 턴별 표	전체 항목 수 = 그대로 + 일반화 + 비공개
같은 파일 (누적)	턴별 누적 그대로율	누적 정의상 값이 내려가지 않음
`kdrop_summary.md`	속성별 평균·중앙값 k 감소량($\log_2$), `k_raw` ≤ 5 비율, 개인정보·의료정보 구분	손계산 한두 턴과 일치
`latency_summary.md`	갈래별 턴 1 이상 중앙값, IQR, p95, p95 신뢰구간, C1−C0 차이	실행 A·B 두 블록이 모두 포함됨
`sweep_summary.md`	기준별(5·3, 10·5, 20·10) C1S·C1 의 그대로율·일반화율·비공개율·평균 유용성	기준 5·3 행이 실행 A 결과와 같음
검증 메모	손계산 대조, 위반율 대조 결과, 이상 턴 목록	불일치가 있으면 턴 번호와 함께 기록
실행 기록	각 실행 폴더의 `run_meta.json` 의 `git_commit`, 시작 시각, 사용 명령	네 폴더 모두 커밋 해시가 있음

9. 이 패키지에 없는 일 (우리 쪽에서 처리)

개발자는 아래 일을 하지 않습니다. - 발표 자료 수치 갱신 (결과 ③, 결과 ⑥, 민감도 표) - 출처 확인: Gartner 2028 수치의 발표 날짜, 희귀질환관리법 2 만명 기준의 원문 대조 - 배경 슬라이드의 “세 겹의 방어” 문구 교체 여부 결정